Pantry

A kitchen inventory and expiry tracker, built and shipped with a full DevOps pipeline

Jils Patel · github.com/Jils31/pantry

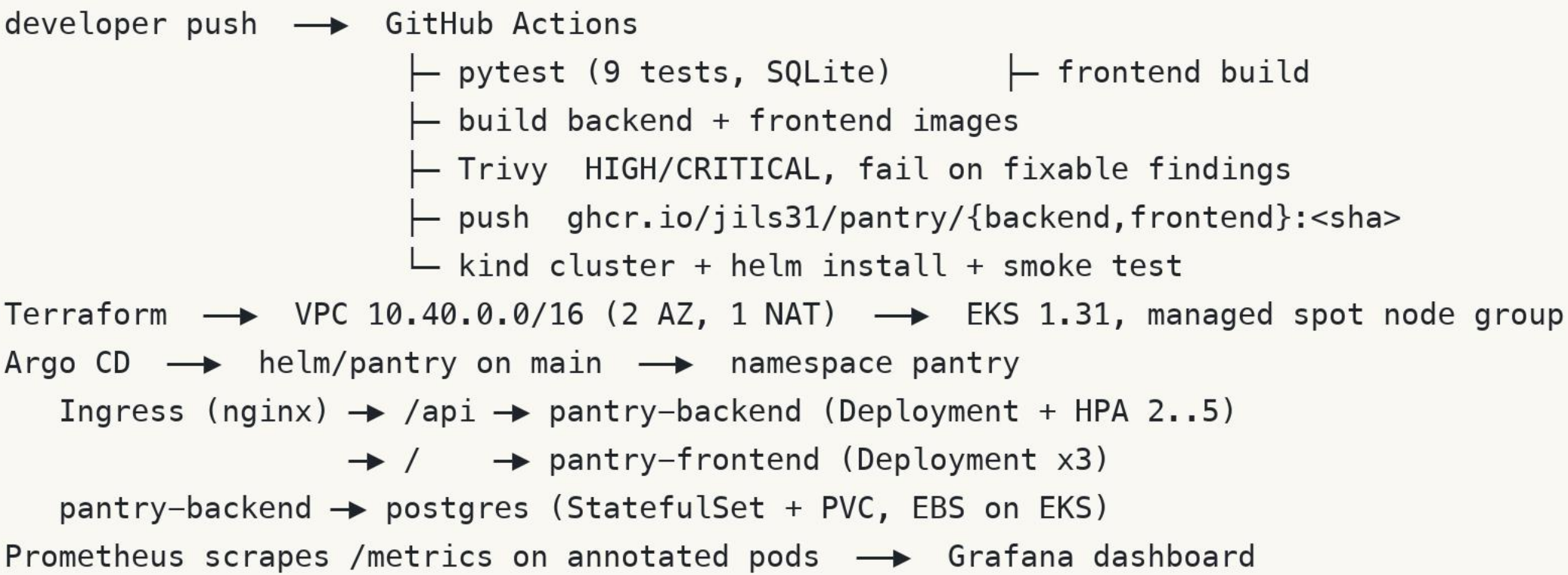
What it does

- Tracks what is in the kitchen: name, category, quantity, unit, location, expiry date
- Flags items that are running low and items expiring within a week
- Consume endpoint decrements stock and refuses to overdraw (HTTP 409)
- Stats endpoint feeds the dashboard counters
- React UI talks to a FastAPI backend over /api; Postgres holds the data

Stack

- Backend: Python 3.12, FastAPI, SQLAlchemy 2, Alembic migrations, psycopg 3
- Frontend: React 18 + Vite, served by nginx-unprivileged on port 8080
- Database: PostgreSQL 16 as a StatefulSet with a persistent volume
- Containers: multi-stage images, non-root users, Trivy scanned, pushed to GHCR
- Platform: Kubernetes (Minikube and EKS), Helm, Argo CD, Prometheus, Grafana
- Infrastructure: Terraform with the community VPC and EKS modules

Architecture



CI/CD pipeline

- test: pytest against SQLite, so the job needs no database service
- frontend: npm ci and vite build
- images: matrix over backend and frontend; build, Trivy, push to GHCR tagged with the commit SHA
- deploy: kind cluster, helm install with the fresh images, curl /ready and /api/items
- Earlier runs failed on Alpine OpenSSL and Starlette CVEs; fixed with base-image updates and a FastAPI bump

Images: ghcr.io/jils31/pantry/backend and ghcr.io/jils31/pantry/frontend

Terraform and EKS

- VPC module: 10.40.0.0/16, two public and two private subnets, one NAT gateway
- EKS module: cluster 1.31, add-ons coredns, kube-proxy, vpc-cni, aws-ebs-csi-driver
- Managed node group on spot t3.small (the account is on the Free Tier plan)
- Node role carries AmazonEBSCSIDriverPolicy so the Postgres PVC binds
- Region ap-south-1; terraform.tfvars is gitignored, an example file is committed

Lesson learned: without the EBS CSI add-on a PVC on EKS stays Pending forever

Kubernetes and Helm

- One chart, helm/pantry, with values files for dev, prod and EKS
- Backend: Deployment, readiness on /ready, liveness on /health, HPA 2 to 5 pods at 60% CPU
- Frontend: Deployment with the nginx config from a ConfigMap
- Postgres: StatefulSet with a volumeClaimTemplate; credentials in a Secret
- Ingress routes /api to the backend and everything else to the frontend
- Resource requests and limits on every container

GitOps with Argo CD

- k8s/argocd-application.yaml points Argo CD at helm/pantry on main
- Automated sync with prune and self-heal: change values.yaml, push, the cluster follows
- Replica count on the backend Deployment is ignored because the HPA owns it
- Manual kubectl changes get reverted, which is the point

Observability

- Backend exposes /metrics with prometheus_client
- pantry_http_requests_total by method, route and status; request duration histogram; in-flight gauge
- Business counters: pantry_items_added_total and pantry_consumed_total by unit
- Prometheus discovers pods through the prometheus.io/scrape annotation
- Grafana dashboard is provisioned from a ConfigMap: request rate, p95 latency, 5xx share, HPA replicas

Troubleshooting drill

- Fault 1: backend pointed at a database host that does not exist; found through pod status, events and logs
- Fault 2: Ingress /api path names a Service that does not exist; found by walking Ingress to Service to Endpoints
- Method: status, events, logs, then the path between components, before changing anything

Summary

- Working app with tests, scanned images and a green pipeline on every push
- Reproducible infrastructure on AWS, created and destroyed with Terraform
- Declarative deploys through Helm and Argo CD, scaling through the HPA
- Metrics, dashboards and a documented way to debug when it breaks
- Repository: github.com/Jils31/pantry